

I: Impossible Primebox

La Primebox imposible

Nombre del archivo fuente (*Source file name*): `impossible.c`, `impossible.cpp`, `impossible.java` o `impossible.py`

Autor (*Author*): Julián Badillo

«Tengo un reto para ti», dice Danny O’sea, famosa estafadora profesional y especialista en atracos, «necesito que abras esta caja fuerte muy segura, llamada la *Primebox*. Te lo prometo, la paga vale la pena».

El rumor en la ciudad es que Danny O’sea está organizando el atraco más impresionante de su época: robar el Ultra Luxurious McGuffin, actualmente exhibido en el Extremely Securest Room de la Very Secure Gallery y, para colmo, guardado bajo llave dentro de la Primebox.

«He conseguido, a un gran costo, información crucial sobre cómo funciona la Primebox», te dice.

La *Primebox* está compuesta por L cerrojos primos (*primelocks*), numerados $0, 1, \dots, L - 1$. El cerrojo primo i es un fragmento de código de programación de la forma:

```
prlck i:
x = x[ * Ai][ + Bi]
if Pi div x jumpto {prlck ji | end} else jumpto {prlck mi | end}
```

donde A_i y B_i son enteros positivos y P_i es un número primo ($0 \leq i, j_i, m_i < L$). Las expresiones entre corchetes son opcionales. De las expresiones entre llaves separadas por el carácter « | », se debe elegir exactamente una. La notación $s \text{ div } t$ significa « s divide a t ».

El cerrojo primo i tiene un *número de activación* (*activation number*) $K_i \geq 0$, cuyo significado quedará claro a continuación. Los cerrojos están dispuestos secuencialmente y se activan (es decir, se ejecutan) empezando por el cerrojo primo 0 y con un valor entero inicial $x \geq 0$. La ejecución de un cerrojo primo se realiza de la «manera natural» (por favor, no preguntes qué significa «natural»...) y todo el proceso termina cuando se ejecuta cualquiera de las instrucciones `jumpto end`.

Para abrir toda la Primebox, el i -ésimo cerrojo primo debe ser *activado* exactamente K_i veces. Puede haber varios números iniciales que activen los cerrojos primos de acuerdo con sus números de activación correspondientes. La *clave* (*key*) de la Primebox es el mínimo de ellos.

«¡Ah! No te preocupes por eso, conseguí un dato más», dice Danny O’Sea, «la clave sí existe y es menor que 50 000».

Tu trabajo, dado el código fuente de los cerrojos primos y sus respectivos números de activación, es descifrar la clave. Una jugosa recompensa te espera.

Por ejemplo, considera una Primebox definida con los siguientes cerrojos primos:

```
prlck 0:
x = x * 2 + 3
if 3 div x jumpto prlck 2 else jumpto prlck 1
prlck 1:
x = x * 4 + 1
if 5 div x jumpto prlck 0 else jumpto prlck 2
prlck 2:
x = x + 5
if 7 div x jumpto prlck 1 else jumpto end
```

y los números de activación $K_0 = 1$, $K_1 = 2$ y $K_2 = 2$. Puedes comprobar que, empezando con $x = 10$, el proceso terminará después de que los cerrojos primos se hayan activado según lo pedido. Sin embargo, esto no ocurre con nin-

gún entero menor que 10. Por lo tanto, esta Primebox se abre con la clave 10.

Entrada (Input)

La entrada consiste en varios casos de prueba. Observa que todas las variables, enteros y operadores descritos aquí están separados por un único espacio en blanco. Cada caso de prueba define una Primebox y comienza con un entero L ($1 \leq L \leq 10$), el número de cerrojos primos. Las siguientes $3 \times L$ líneas describen los cerrojos primos en orden secuencial, cada uno siguiendo el formato descrito arriba. Puedes suponer que, para el cerrojo primo i ($0 \leq i < L$), los parámetros involucrados A_i , B_i y P_i son enteros positivos ($0 \leq A_i, B_i, P_i \leq 1\,000$), P_i es un número primo, y todo número de cerrojo primo referenciado está en el intervalo entero $0..L-1$. [N. del T.: el original dice «enteros positivos» pero escribe la cota inferior como $0 \leq$.] El caso termina con una línea que contiene L números K_i ($0 \leq i < L$ y $0 \leq K_i \leq 1\,000$), los números de activación correspondientes a los cerrojos primos dados. La entrada termina con la línea $\emptyset$, que no debe procesarse.

La entrada debe leerse de la entrada estándar.

Salida (Output)

Para cada caso de prueba, imprime una única línea con la clave descifrada correspondiente.

La salida debe escribirse en la salida estándar.

Ejemplo de entrada (Sample Input)	Ejemplo de salida (Sample Output)
<pre>3 prlck 0: x = x * 2 + 3 if 3 div x jumpto prlck 2 else jumpto prlck 1 prlck 1: x = x * 4 + 1 if 5 div x jumpto prlck 0 else jumpto prlck 2 prlck 2: x = x + 5 if 7 div x jumpto prlck 1 else jumpto end 1 2 2 3 prlck 0: x = x + 3 if 71 div x jumpto end else jumpto prlck 1 prlck 1: x = x * 15 if 131 div x jumpto end else jumpto prlck 2 prlck 2: x = x * 24 + 1 if 3 div x jumpto prlck 1 else jumpto prlck 0 7 7 6 0</pre>	<pre>10 255</pre>